

Merbah Bank: Selasih Core Ledger Migration

Detailed Design

Architecture, requirements, and validation criteria for build

Version	1.1
Status	Design phase, build not started
Last updated	2026-09-21 (architecture), consolidated 2026-09-28, revised 2026-10-19
Prepared by	Merbah Bank Core Banking Engineering, Ledger Programme
Accountable executive	Marguerite Liew, Chief Financial Officer
Companion documents	Selasih Conceptual Design v1.3; Chart of Accounts Mapping Workbook; CORAL Interface Catalogue; Records Retention Standard RRS-3

Revision history

Version	Date	Summary
1.0	2026-09-28	Consolidated detailed design for build
1.1	2026-10-19	Revised after the design review: posting path, interest accrual, revaluation rate, close adjustments, archive tiers, reconciliation method and dual-run dates

Table of Contents

1. Purpose and Scope
2. Requirements
3. Foundational Principles
4. Current State: the CORAL Mainframe Ledger
5. Target Architecture
6. Chart of Accounts and Ledger Structure
7. Journal Model and Double-Entry Invariants
8. Posting API and Idempotency
9. Account Processing and the Posting Path
10. Event Bus and Transactional Outbox
11. Amounts, Currencies and Rounding
12. Interest Accrual
13. FX Revaluation
14. End-of-Day Close
15. Regulatory Reporting
16. Manual Journals and Maker-Checker
17. Access Control and Segregation of Duties
18. Audit Trail
19. Retention and Archive
20. Data Model
21. Data Migration of History
22. Dual-Run and Reconciliation
23. Cut-Over Plan, Go/No-Go Gate and Rollback
24. Resilience and Disaster Recovery
25. Observability
26. Confirmed Decisions
27. Pending Backlog
28. Validation and Acceptance Criteria
29. Implementation Readiness Assessment

Changes since version 1.0

- Sections 9.1, 9.2 and 20: account rules are now checked by the Posting Service when a journal is accepted, against a new `account_limit` table in the Journal Store; the `rejected_leg` table is removed.
 - Sections 12.2 and 12.4: a failed accrual run resumes where it stopped, and accrual and reversal journals have new idempotency keys.
 - Section 2.1 (FR-5) and Section 13: the rate used for month-end revaluation is defined.
 - Sections 14.3 and 17.1: close adjustments are approved by a second user before they post.
 - Section 2.2 (NFR-4), Section 19 and Section 28.2: archived records move to S3 Glacier Instant Retrieval for seven years and to S3 Glacier Deep Archive after that, and the retrieval times are restated per tier.
 - Section 22.2: the daily reconciliation compares account balances and matches individual postings.
 - Section 30: the dual-run dates.
-

1. Purpose and Scope

This document describes the architecture of Selasih, the event-sourced general ledger service that will replace CORAL, the mainframe batch ledger that has held Merbah Bank's books since 1998. It is written to a level of technical detail sufficient for an engineering team to begin implementation. Section 29 gives an explicit assessment of where that is and is not yet true.

Merbah Bank is a Singapore-incorporated retail bank that has operated as a digital bank since 2023. It serves about 1.3 million customers holding about 2.8 million accounts: Merbah Save savings accounts, Merbah Flex current accounts, Merbah Term fixed deposits, Merbah Kredit personal loans and Merbah Global multi-currency wallets in SGD, USD, MYR, IDR, EUR and AUD. Today every one of these products posts to CORAL, which updates balances in a nightly batch and serves intraday balances from a memo-post layer.

Selasih provides one posting API for every product system and channel; real-time, authoritative account balances; double-entry journals that are immutable once accepted; daily interest accrual and month-end FX revaluation; an end-of-day close that produces the trial balance and the data for regulatory returns; maker-checker control over manual journals; and a tamper-evident audit trail.

In scope

The Selasih ledger service, the migration of seven years of posting history from CORAL, a dual-run period in which both ledgers process the same business, the cut-over from CORAL to Selasih, the end-of-day close, the regulatory reporting extracts, and disaster recovery across two cloud regions. Volumes: an average of 1.6 million posting legs a day, about 700,000 journals, with a payday peak of 450 journals per second.

Out of scope

The product systems themselves (deposits, lending, cards and the payments hub), which keep their own customer-facing state and call Selasih to post. The general ledger consolidation package used by Group Finance, which receives a daily trial balance file. Customer statements, which are produced by the Statements Service from Selasih's event stream and specified separately.

Readers

Core Banking Engineering, Finance (Financial Control, Financial Reporting, Treasury), Regulatory Reporting, Technology Risk, Internal Audit and the cut-over team.

2. Requirements

2.1 Functional requirements

ID	Requirement
FR-1	Every journal accepted by Selasih balances: in each currency, the sum of its debit legs equals the sum of its credit legs.
FR-2	Every posting request carries an idempotency key; a repeated request with the same key never creates a second journal.
FR-3	Each account's balance is available to product systems in real time, reflecting every accepted journal.
FR-4	Interest is accrued daily for every interest-bearing account and capitalised or paid on the product's schedule.
FR-5	Non-SGD monetary positions are revalued to SGD at each month end at the Treasury closing rate defined in Section 13, with the difference posted to unrealised FX gain or loss.
FR-6	The end-of-day close produces a trial balance and the regulatory reporting extracts for the business day.
FR-7	Selasih produces the data for the capital adequacy return, the liquidity coverage ratio return and the monthly statistical returns.
FR-8	Every manual journal is entered by one authorised user (maker) and approved by a different authorised user (checker) before it posts.
FR-9	Seven years of CORAL posting history are migrated into Selasih and are queryable by account and date.
FR-10	During the dual-run, CORAL and Selasih are reconciled every business day.
FR-11	The cut-over has a go/no-go gate and a rehearsed rollback to CORAL.
FR-12	Every journal, approval and change to reference data is recorded in a tamper-evident audit trail.
FR-13	Ledger and audit records are retained for the period set by RRS-3 and can be produced on request.

2.2 Non-functional requirements

ID	Requirement
NFR-1	Posting API latency of 150 ms or less at the 99th percentile, measured at the API, at a sustained 1,000 journals per second.
NFR-2	Posting API availability of 99.95 percent per calendar month, excluding the published maintenance window.
NFR-3	Recovery point objective of zero for every journal acknowledged to a caller, for any failure up to and including the loss of the primary region; recovery time objective of one hour.
NFR-4	Any ledger or audit record up to seven years old can be produced for a regulator or auditor request within four hours, and any older record within the retention period within 48 hours.
NFR-5	The ledger scales to accommodate the bank's future growth without redesign.
NFR-6	The end-of-day close completes, with the trial balance published, by 02:00 SGT.
NFR-7	No single person can both create and approve a manual journal, or change reference data without a second approval.
NFR-8	Any alteration or deletion of a posted journal or audit record is detectable.

3. Foundational Principles

P1. The journal is the record. Account balances, the trial balance and every report are derived from journals. Nothing changes a balance except an accepted journal.

P2. A journal is all or nothing. A journal is accepted in full or not at all; no part of a journal can take effect without the rest.

P3. Immutability. An accepted journal is never updated or deleted. Corrections are new journals that reverse or adjust.

P4. Idempotency at the boundary. Every caller supplies a key; the ledger, not the caller, guarantees that a key posts at most once.

P5. Integer minor units. Amounts are integers in the currency's minor unit. Floating point is never used for money.

P6. Segregation of duties. The people who build and operate the ledger cannot post to it, and the people who post manual entries cannot approve their own.

P7. Prove before switch. Selasih takes over from CORAL only after it has produced the same books as CORAL over a sustained dual-run.

4. Current State: the CORAL Mainframe Ledger

CORAL runs on the bank's z/OS mainframe in the Tai Seng data centre. Product systems send postings to CORAL's online interface over IBM MQ during the day. CORAL validates each posting, writes it to a memo-post file and updates a memo balance used for available-balance checks. The nightly batch, which starts at 22:00 SGT, applies the day's memo posts to the master balance files, runs interest accrual, produces the trial balance and writes the extract files for regulatory reporting. The batch typically completes by 01:30 SGT.

CORAL accepts back-valued postings up to 30 days old, such as interbank returns and card refunds that carry their original value date, and recomputes accrued interest for each affected day of the affected account.

CORAL holds posting history for seven years online in VSAM files; older history is on tape. Its chart of accounts has 4,100 general ledger accounts, of which about 1,900 are active.

The reasons for replacement are set out in the conceptual design: the mainframe support contract ends in 2028, intraday balances are memo balances rather than ledger balances, every new product needs COBOL changes, and the batch window has grown by 40 minutes in three years.

5. Target Architecture

Selasih runs on AWS in ap-southeast-1 (Singapore), across three Availability Zones, with a disaster recovery footprint in ap-southeast-3 (Jakarta). It has five components:

- **Posting Service.** Receives posting requests over HTTPS from product systems, validates them, enforces idempotency and writes accepted journals to the Journal Store. Runs on Amazon EKS.
- **Journal Store.** Aurora PostgreSQL 16 cluster holding journals with their idempotency keys, legs and the outbox. Writer plus two readers.
- **Account Processors.** Consumers of the ledger. Legs topic that maintain authoritative account balances in the Balance Store (a second Aurora PostgreSQL cluster) and enforce account rules.
- **Close and Reporting Service.** Runs the accrual, revaluation and end-of-day close jobs and builds the regulatory reporting extracts.
- **Consoles.** The Journal Console (manual journals with maker-checker), the Close Console (close monitoring and adjustments) and the Reference Data Console (chart of accounts, products, rates).

The event bus is Amazon MSK (Apache Kafka). Users sign in to the consoles through the bank's identity provider (OIDC), with group membership mapped to Selasih roles. Services authenticate to each other and to MSK with IAM.

Figure 1. Product systems call the Posting Service; journals are written to the Journal Store with an outbox; the outbox relay publishes legs and journal events to MSK; Account Processors apply legs to the Balance Store; the Close and Reporting Service reads both stores; the consoles sit behind the identity provider.

6. Chart of Accounts and Ledger Structure

Selasih keeps two levels of account. **General ledger (GL) accounts** follow the bank's chart of accounts, restructured during the programme from CORAL's 4,100 accounts to 1,240 accounts in a five-segment code: entity, natural account, product, currency and cost centre. **Customer accounts** are sub-ledger accounts, each attached to exactly one GL control account (for example, all Merbah Save balances in SGD roll up to GL 2101-SAV-SGD).

The rule that the sum of customer account balances equals the balance of their control account holds by construction: a leg posted to a customer account carries its control account and is counted in both. The Chart of Accounts Mapping Workbook maps every CORAL account to one Selasih account; it is signed off by Financial Control before data migration begins.

6.1 Suspense and clearing accounts

Every product system has its own suspense account per currency. The Posting Service never posts to suspense on its own initiative; a product system that cannot identify the beneficiary account of an incoming credit posts it to its suspense account and clears it with a later journal. Suspense balances older than five business days are reported daily to Financial Control, and the close refuses to publish the trial balance if a suspense account flagged as "must clear daily" (the FAST and GIRO inward clearing accounts) is not zero.

6.2 Account lifecycle

Customer accounts are opened in Selasih by the product system with a `POST /v1/accounts` call that names the product, the currency and the control account; Selasih assigns no account numbers itself. Closing an account requires a zero balance and no active holds. Reference data changes to GL accounts (new account, change of mapping, deactivation) go through the Reference Data Console with maker-checker approval and take effect from a stated business date, never retroactively.

7. Journal Model and Double-Entry Invariants

A **journal** is one business event: a transfer, a card settlement, an accrual, a revaluation or a manual entry. It has a journal ID, a source system, the source system's reference, a business date, a value date, a description and two or more **legs**. Each leg names one account, one currency, a direction (debit or credit) and an amount in minor units.

Selasih enforces double entry at three points:

1. **On request.** The Posting Service rejects a request whose legs do not balance in each currency. A cross-currency journal, such as a customer converting SGD to USD, has four legs: the SGD debit to the customer and credit to the FX position account in SGD, and the USD debit to the FX position account and credit to the customer in USD. Each currency balances on its own.
2. **On commit.** The `leg` table has a deferred constraint trigger that, at commit, sums each journal's legs by currency and aborts the transaction if any currency is out of balance. A defect in the Posting Service therefore cannot commit an unbalanced journal.
3. **At close.** The end-of-day close recomputes, from the legs, the sum of debits and credits per currency for the day, adds it to the cumulative totals recorded by the previous close run, and refuses to publish the trial balance if the day's or the cumulative difference is not zero.

Example: a customer pays SGD 120.50 from Merbah Save to an external account through FAST. Journal J1 debits the customer's savings account 12050 and credits the FAST settlement account 12050. If the payment is returned, journal J2 reverses J1 with the same two legs in the opposite directions; J1 is never altered.

Example with fees: a USD 2,000.00 remittance with a USD 15.00 fee. The customer account is debited 201500; the correspondent settlement account is credited 200000; fee income is credited 1500. $200000 + 1500 = 201500$.

8. Posting API and Idempotency

`POST /v1/journals` takes the journal, the source system (from the caller's IAM identity, not from the body) and an Idempotency-Key header. The Journal Store holds a unique constraint on (`source_system`, `idempotency_key`) in the `journal` table itself, and the key is kept for the life of the journal, so there is no expiry window in which a late retry could post twice.

The Posting Service inserts the journal, its legs and its outbox rows in one transaction. If the insert fails on the unique constraint, the service reads the existing journal and compares a SHA-256 hash of the canonical request body with the stored hash:

- same hash: the request is a retry, and the service returns the original response (201 with the original journal ID);
- different hash: the key has been reused for a different journal, and the service returns 409 Conflict without posting.

Because the constraint is enforced by the database, two concurrent requests with the same key cannot both succeed, whichever Posting Service pod receives them. Callers are told to generate keys from their own business reference (for example the payment ID), so that a caller restart regenerates the same key.

9. Account Processing and the Posting Path

The Journal Store guarantees that each journal balances; the Account Processors keep each account's balance and

apply each account's rules.

9.1 Flow

1. The Posting Service validates the request (shape, balance, accounts exist, reference data) and checks the account rules of Section 9.2 for every account the journal touches, inserts the journal and returns 201 Created with the journal ID.
2. The outbox relay publishes each leg to the `ledger.legs` topic, keyed by account number, and a journal event to `ledger.journals`.
3. The Account Processor that owns the leg's partition applies it to the account's row in the Balance Store and records the leg ID in the row's applied-leg set, so a redelivered leg is not applied twice.

9.2 Account rules

The account rules depend on the account's current state: a closed account accepts no postings, a frozen account (court order, deceased estate, fraud hold) accepts credits only, and a debit may not take a non-overdraft account below zero. The Posting Service checks these rules for every leg of a journal in the same transaction that inserts the journal: for each customer account the journal touches, taken in account order, it locks the account's row in the `account_limit` table in the Journal Store, checks the account's status and, for a debit to a non-overdraft account, that the committed balance less active holds covers the debit, and applies the leg to the committed balance before commit. If any leg fails a rule, the whole journal is refused with 422 and nothing is written. The Account Processors apply the legs of accepted journals and never reject a leg.

9.3 Throughput

`ledger.legs` has 96 partitions. At the NFR-1 rate of 1,000 journals per second, with an average of 2.3 legs per journal, the processors apply about 2,300 legs per second, or about 24 per partition per second; each processor batches its Balance Store updates every 50 ms. Measured end-to-end lag from journal commit to balance update in the prototype was 180 ms at the 99th percentile.

9.4 Available balance for product systems

Product systems read balances from `GET /v1/accounts/{id}/balance`, served from the Balance Store readers. The response carries the ledger balance and the available balance (ledger balance minus active holds placed by the cards system), and `GET /v1/journals/{id}` reports whether each leg of a journal has been applied, so a caller that has just posted can wait until its own legs are reflected.

9.5 Holds

The cards system places holds when a card authorisation is approved and releases them when the matching settlement posts or the hold expires after seven days. Holds reduce the available balance but never the ledger balance, and they are not journals; they live in the Balance Store's `hold` table and are exposed through the balance endpoint. Because holds are memo items, a released or expired hold has no accounting effect and needs no reversal.

10. Event Bus and Transactional Outbox

Selasih never writes to the database and to MSK as two separate steps. Each journal transaction inserts outbox rows (one per leg for `ledger.legs`, one per journal for `ledger.journals`) in the same transaction as the journal. The outbox relay reads committed rows in order of their sequence number, publishes them with the idempotent producer enabled, and marks them published; a relay that stops between publishing and marking republishes, and every consumer deduplicates by the event ID carried in the message.

Ordering: legs are keyed by account number, so all legs for one account go to one partition and are applied by one Account Processor. An account's balance is the sum of its legs, so it does not depend on the order in which they arrive. Journal events are keyed by journal ID. No consumer depends on ordering across accounts.

Topics are replicated three times across the three Availability Zones with `min.insync.replicas` of 2 and `producer.acks=all`. Retention on `ledger.legs` and `ledger.journals` is 14 days; the Journal Store, not the topic, is the system of record, and any consumer can be rebuilt by replaying from the Journal Store through a backfill relay.

Downstream consumers are the Account Processors, the Statements Service, the fraud monitoring feed and the data lake loader. Each consumer has its own consumer group and commits offsets only after its own write has committed.

11. Amounts, Currencies and Rounding

All amounts are stored as `bigint` in the currency's minor unit, with the exponent taken from ISO 4217 (SGD, USD, MYR, IDR, EUR and AUD all have two decimal places). The largest single balance the bank holds, the central bank reserve account, is about SGD 4.2 billion, or 420 billion minor units, far within the range of a 64-bit integer.

Calculations that produce fractions of a minor unit (interest, FX conversion, fee percentages) are carried out in `numeric(38, 12)`. The posted amount is rounded half-even to the minor unit, and for interest the unrounded remainder is carried forward: each day's posted accrual is the cumulative accrual for the month, rounded, minus the amount already posted for the month. Over a month the posted total therefore differs from the exact total by less than one minor unit.

FX conversions record the rate used, its source and its timestamp on the journal, so every converted amount can be recomputed.

12. Interest Accrual

12.1 Method

Interest accrues daily on the end-of-day ledger balance. SGD, MYR and AUD products use Actual/365 Fixed; USD and EUR products use Actual/360; IDR uses Actual/365 Fixed. Tiered savings rates (Merbah Save pays 1.20 percent on the first SGD 50,000 and 2.10 percent above it) are applied per tier.

12.2 The accrual run

The accrual run starts at 23:30 SGT, after the business day closes for postings (Section 14), and works through the 1.9 million interest-bearing accounts in account-number order, in 64 parallel workers. For each account it computes the day's accrual (Section 11), and posts one journal: debit interest expense (deposits) or interest receivable (loans), credit accrued interest payable (deposits) or interest income (loans), with the customer account named on the payable or receivable leg.

A run that fails part-way resumes from the first account it has not yet posted, and each accrual journal's idempotency key is ACR- followed by the account number and the accrual date, so a restarted run cannot post one account's accrual for one date twice. The Posting API's 409 response for a key that already holds a journal is treated as confirmation that the account's accrual for that date is posted, and the run moves on to the next account.

12.3 Capitalisation

On the product's capitalisation date (month end for Merbah Save, maturity for Merbah Term), the accrued interest payable for the account is moved to the customer account in one journal.

12.4 Back-dated rate changes

When Treasury enters a rate change with an effective date in the past, it starts a re-accrual for the affected product and dates: for each affected account and date, the run posts a reversal of the original accrual journal and then a new accrual journal at the corrected rate. Reversal journals carry the idempotency key REV- followed by the journal ID of the journal they reverse.

12.5 Loans in arrears

A Merbah Kredit loan that is 90 days past due is placed in non-accrual status by the lending system, which sends a status change to Selasih. From the next business day the accrual run skips the account, and the lending system tracks contractual interest in memo form. Interest already accrued but unpaid when the loan enters non-accrual is reversed by a journal raised by the lending system, so that income is not recognised on interest the bank does not expect to collect.

12.6 Timing

At 1.9 million accounts and 64 workers, the run posts about 900 journals per second for its 35 minutes. It runs through the same Posting API as every other caller and is subject to the same validation; the accrual worker's IAM role may post only to the interest GL accounts and accrued interest accounts listed for it in reference data.

13. FX Revaluation

Merbah Global wallets and the bank's nostro accounts hold USD, MYR, IDR, EUR and AUD. Monetary positions in those currencies are carried at historical SGD equivalents through the month.

The Treasury closing rate for a currency is the mid rate against SGD that Treasury fixes at 17:00 SGT on the last business day of the month and publishes to the `ref.fx_rate` table through the rate service; the revaluation job reads the rate with that business date and refuses to run if any currency's rate is missing.

At each month end, after the last business day's close, the revaluation job computes for every non-SGD monetary GL account the SGD equivalent of its balance at the Treasury closing rate, compares it with the carried SGD equivalent and posts the difference to unrealised FX gain or loss (GL 7310) against an FX revaluation reserve account per currency. On the first business day of the next month the job posts the exact reversal, so the following month's revaluation starts from historical equivalents again and the unrealised result is never counted twice.

Non-monetary items (fixed assets, prepaid expenses in foreign currency) are not revalued. Revaluation runs per GL account, not per customer account; customer wallet balances are always shown in their own currency.

14. End-of-Day Close

14.1 Business day

The Selasih business day runs from 00:00:00 to 23:29:59 SGT. At 23:30 the Close and Reporting Service rolls the business date: postings arriving from then on carry the next business date, and the close for the day that has just ended begins. The close's own accrual and revaluation journals carry the business date being closed. Once a business day is closed, its balances are final: a posting that arrives after the close carries the next business day's value date, whatever value date the source system sent.

14.2 Close steps

1. Roll the business date (23:30).
2. Wait for the Account Processors to apply every leg committed before the roll (the close waits until every outbox row committed before the roll is published and each processor has consumed its partition past the last of those rows).
3. Run interest accrual (Section 12), typically 35 minutes.
4. On the last business day of the month, run FX revaluation (Section 13).
5. Recompute the double-entry checks (Section 7, point 3).
6. Produce the trial balance per entity and currency, and the regulatory reporting extracts (Section 15).
7. Publish the trial balance file to Group Finance's consolidation package and mark the day closed.

The close typically completes by 00:50 SGT, an hour and ten minutes inside NFR-6.

14.3 Close adjustments

Some corrections must be made inside the close window: an accrual on a product whose rate was mis-keyed that day, a misposted fee batch, a suspense balance that must be cleared before the trial balance publishes. Close adjustments raised during the close window are entered by a Financial Controller in the Close Console and approved by a second Financial Controller or by the Head of Financial Reporting before they post; the close rota always names two Financial Controllers on call, so an approval is available inside the window. Each adjustment is recorded in the audit trail with the user's identity and a mandatory reason, and Financial Control reviews the list of adjustments the next morning.

15. Regulatory Reporting

The Close and Reporting Service produces three groups of extracts from the closed day's balances, each with a lineage record that links every reported figure to the GL accounts and journals behind it:

- **Capital adequacy.** Balances by exposure class and risk-weight bucket for the capital adequacy return under MAS Notice 637, produced at each quarter end and monthly for internal monitoring.
- **Liquidity.** Balances by liquidity category and maturity bucket for the liquidity coverage ratio return under MAS Notice 649, produced daily for internal monitoring and monthly for submission.
- **Statistical returns.** Monthly balance sheet and income data for the MAS 610 returns.

The extracts are loaded into the Regulatory Reporting team's reporting tool, which applies the regulatory mappings and produces the returns; the mappings themselves are outside Selasih. Each extract carries the business date, the trial balance checksum and the close run ID, and the reporting tool refuses an extract whose checksum does not match the published trial balance.

15.1 Restatement

When a closed period must be restated for a return (for example, after an audit adjustment), the restatement is a new set of journals dated in the current period with a reference to the period they correct; the returns for the corrected period are then rebuilt by the reporting tool from the original extract plus a restatement extract that lists those journals. The original extracts are never overwritten, so the figures as first submitted can always be reproduced.

15.2 Timing

The daily liquidity extract is available by 01:00 SGT for Treasury's morning liquidity review. Month-end extracts are produced after the revaluation run and are available by 02:00 SGT on the first calendar day of the next month; the reporting tool's submission calendar allows several business days after that before any return is due.

16. Manual Journals and Maker-Checker

Manual journals are entered in the Journal Console by users in the `journal-maker` role and approved by users in the `journal-checker` role. A user cannot hold both roles, and the console refuses approval by the user who created the journal even if role assignments change. Journals above SGD 1,000,000 require a second checker from the `journal-senior-checker` role. An approved journal is submitted to the Posting Service under the Journal Console's service identity with the maker's and checkers' identities in its metadata.

A pending manual journal does not affect any balance. A rejected journal is kept with the checker's reason. Journals pending for more than three business days expire.

16.1 Recurring journals

Recurring manual journals, such as monthly accruals for rent and service contracts, are set up once as templates by a maker and approved by a checker; each generated instance posts on its scheduled date without a further approval, but any change to the template's accounts, amount or schedule is a new template that needs approval. Templates expire after twelve months and must be re-approved.

16.2 Reversals of manual journals

A manual journal can be reversed only through the Journal Console, by a new manual journal that references it; the reversal goes through the same maker-checker flow as the original.

Bulk manual journals (up to 5,000 lines) are uploaded as CSV files by a maker, validated in full before submission for approval, and approved or rejected as a whole; a file whose debits total more than SGD 1,000,000 needs the second checker as well.

17. Access Control and Segregation of Duties

17.1 People

Console users sign in through the bank's identity provider with phishing-resistant MFA. Roles are assigned through identity provider groups, which are requested through the access request workflow and approved by the role owner (Financial Control for journal roles, Core Banking Engineering for operational roles). Role assignments are recertified quarterly.

Role	Can
journal-maker	Create manual journals
journal-checker	Approve or reject manual journals
journal-senior-checker	Second approval above SGD 1,000,000
financial-controller	Run and monitor the close; enter close adjustments and approve those entered by others
refdata-maker / refdata-checker	Propose and approve reference data changes
ledger-operator	Restart jobs, view metrics and logs; no posting rights
auditor	Read-only access to journals and the audit trail

Engineers have no standing access to production data. Break-glass access to the Journal Store is granted for four hours through the privileged access workflow, with session recording, and is reviewed by Technology Risk.

17.2 Services

Each product system calls the Posting Service under its own IAM role, and the Posting Service records that identity as the journal's source system; a product system can post only to the GL control accounts listed for it in reference data. To keep onboarding simple, every workload in the ledger AWS account runs under the shared `ledger-workload` IAM role, which is granted `kafka-cluster:WriteData` on all topics matching `ledger.*`. The workloads in the ledger account are the Posting Service, the outbox relay, the Account Processors, the Close and Reporting Service, the consoles' back ends, the reconciliation engine and the history migration loader.

18. Audit Trail

Every journal, every manual journal approval or rejection, every reference data change, every role-relevant console action and every close run writes an audit record to the `audit_event` table in the same transaction as the change it records; a journal's audit record carries the SHA-256 hash of the journal and its legs, so a journal altered afterwards no longer matches its record. The table is append-only: the application's database role has INSERT and SELECT on it and nothing else, and the table owner role is not used by any service.

A chaining job, running continuously, takes each audit record not yet chained and appends to the `audit_chain` table the SHA-256 hash of that record combined with the hash of the previous chain entry, forming a hash chain; only the chaining job's role can insert into `audit_chain`, and no role can update or delete its rows. Every five minutes, a job writes the latest chain hash to an S3 bucket with Object Lock in compliance mode and a ten-year retention, in a separate AWS account owned by Technology Risk. Internal Audit's verification tool recomputes the chain from the tables and compares it with the stored hashes; any altered, deleted or inserted record breaks the chain from that point. The stored hashes themselves cannot be overwritten or deleted by anyone, including the account root user, until their retention ends.

Audit records are exported monthly to the archive (Section 19).

19. Retention and Archive

RRS-3 requires ledger records (journals, legs and the trial balance) and audit records to be kept for ten years from the end of the financial year to which they relate.

The Journal Store keeps 13 months of journals online. Each month the archive job writes the month that has passed out of the online window to Parquet files in S3, partitioned by business date and account, with a manifest and SHA-256 checksums, moves them to S3 Glacier Instant Retrieval after 30 days, and moves them to S3 Glacier Deep Archive

when they are seven years old. The archive bucket has Object Lock in compliance mode, and each file's retention ends ten years after the end of the financial year of the newest record it holds.

Archived records are retrieved through the Archive Service, which takes an account and a date range. Records up to seven years old are read directly from S3 Glacier Instant Retrieval, which serves objects in milliseconds, so the four-hour limit of NFR-4 is met for them. For an older record the Archive Service requests a Standard retrieval from S3 Glacier Deep Archive, which completes within 12 hours, inside the 48 hours NFR-4 allows for those records. Retrieved files are restored for seven days, and every retrieval is recorded in the audit trail.

20. Data Model

Journal Store (Aurora PostgreSQL 16):

Table	Key columns	Notes
journal	journal_id (uuid, PK), source_system, idempotency_key, request_hash, business_date, value_date, source_ref, kind, created_at	Unique (source_system, idempotency_key); not partitioned
leg	leg_id (bigint, PK), journal_id, business_date, account_id, control_account_id, currency, direction, amount_minor	Deferred balance trigger per journal and currency; partitioned by month on business_date
outbox	seq (bigint, PK), topic, key, payload, published_at	Relay reads by seq
account_limit	account_id (PK), status, overdraft_allowed, committed_balance_minor, holds_minor	Locked and updated by the Posting Service for every leg to a customer account; holds mirrored from the cards system
audit_event	audit_id, actor, action, subject, subject_hash, at	Append-only
audit_chain	chain_seq (bigint, PK), audit_id, prev_hash, hash	Insert-only, written by the chaining job
close_run	business_date, run_id, status, tb_checksum, cumulative_totals	One per business day

Balance Store (Aurora PostgreSQL 16):

Table	Key columns	Notes
account_balance	account_id (PK), currency, ledger_balance_minor, last_seq, status	Updated by Account Processors
applied_leg	account_id, leg_id	Primary key on both; deduplicates redelivery
hold	hold_id, account_id, amount_minor, expires_at	Placed by the cards system

Reference data (accounts, products, rates, GL mappings) lives in the Journal Store under the `ref` schema, versioned with effective dates.

21. Data Migration of History

21.1 What is migrated

Seven years of posting history (business dates from 1 April 2020 to cut-over), the chart of accounts mapping, every customer and GL account with its status, and the opening balances at cut-over. History before April 2020 stays on CORAL's tapes, which are retained under RRS-3 and read through the existing tape retrieval process.

21.2 Method

The Pelita Extract utility from Rambai Software reads CORAL's VSAM history files and writes fixed-width files, which the history migration loader converts to Selasih journals. Historical journals are loaded with `kind = 'migrated'`, their CORAL reference as `source_ref`, and their original business and value dates; they pass the same balance checks as live journals (Section 7) and bypass the Account Processors, because historical balances are not replayed.

The seven-year history holds about 2.9 billion posting legs; the loader writes 150,000 legs per second into the partitioned `leg` table, so a full load completes in about 90 minutes, and each migration rehearsal loads the full history in

the Saturday 02:00 to 06:00 window.

21.3 Validation

For every account and every month end in the history, the loader computes the closing balance from the migrated legs and compares it with CORAL's month-end balance file. Any difference is a migration defect and is fixed in the mapping or the loader before the next rehearsal.

22. Dual-Run and Reconciliation

22.1 Dual-run

During the dual-run, CORAL remains the system of record. Product systems keep posting to CORAL; a dual-feed adapter on the MQ bridge copies every posting to Selasih's Posting Service with the CORAL transaction ID as idempotency key. Both ledgers run their full close every night. The dual-run covers two full month-end closes, at least eight weeks, so that month-end accrual capitalisation and FX revaluation are exercised twice.

22.2 Daily reconciliation

The reconciliation engine runs at 03:00 SGT, after both closes. It reads CORAL's account balance file and posting file for the business day, and Selasih's balances and journals for the same day. The daily reconciliation compares the closing balance of every mapped account in each currency held by CORAL and by Selasih, and matches each CORAL posting to the Selasih journal that carries its transaction ID as idempotency key, comparing accounts, amounts and value dates; a day on which every balance agrees to the cent and every posting is matched is a matched day.

A day that does not match is a break. The engine writes a break report, and the Ledger Programme's reconciliation team investigates before 10:00 SGT; the cause is fixed in Selasih or in the mapping, and the day is rerun.

22.3 Reporting

The reconciliation dashboard shows matched and broken days, open breaks and their age. The go/no-go gate in Section 23 depends on it.

22.4 Postings that do not reach Selasih

The dual-feed adapter records every posting it receives from the MQ bridge and the Posting Service's response. A posting that Selasih rejects (for example, because its CORAL account has no mapping) is written to the adapter's exception queue, investigated by the reconciliation team, and resubmitted with the same idempotency key once the cause is fixed. The adapter's daily count of postings received is compared with CORAL's daily posting count; a difference opens an investigation even if the totals match.

22.5 Product system changes

Product systems need no change for the dual-run: they keep sending postings to CORAL's MQ interface. For the cut-over, each product system has a configuration switch that points its posting client at Selasih's Posting Service. The client library, written by the Ledger Programme, maps the CORAL message format to the Selasih request and generates the idempotency key from the product system's own transaction reference.

23. Cut-Over Plan, Go/No-Go Gate and Rollback

23.1 Go/no-go gate

The Steering Committee, chaired by the CFO, decides go or no-go on the Thursday before the cut-over weekend. The criteria are:

1. Twenty consecutive matched business days in the dual-run reconciliation (Section 22.2), with no open break.
2. Three full cut-over rehearsals completed, the last within the planned timeline.

3. History migration validation (Section 21.3) passed with zero differences.
4. The rollback rehearsed end to end at least once.
5. Sign-off from Financial Control, Regulatory Reporting, Technology Risk and Internal Audit.

23.2 Cut-over timeline

The cut-over runs over the weekend of Saturday 3 April 2027.

Time (SGT)	Step
Fri 22:00	CORAL's nightly batch starts; the dual-feed adapter is disabled
Fri 22:40	The batch has applied the day's memo posts; the final balance extract is taken from CORAL's master files
Fri 23:00	Opening balances loaded into Selasih from the extract and checked against the extract's control totals
Sat 00:00	T-0: product systems switched from CORAL's MQ interface to Selasih's Posting Service
Sat 00:15	Smoke tests on live traffic
Sat 06:00	Go-live confirmation call

Postings that reach CORAL's online interface between the start of the batch at 22:00 and the 00:00 switch are held in CORAL's memo-post queue for the following business day's batch, and CORAL's batch schedule is disabled at 00:00. After the switch CORAL is set to read-only and kept available for enquiries for twelve months.

23.3 Rollback

Rollback is possible until 18:00 on the Sunday. If the Steering Committee calls rollback, product systems are switched back to CORAL's MQ interface, CORAL's batch schedule is re-enabled, and the journals that product systems and the Journal Console posted to Selasih after T-0 are extracted and replayed into CORAL through the MQ bridge with their Selasih journal IDs as references; Selasih's own accrual and close journals are not replayed, because CORAL's batch computes its own. The rollback has been designed so that CORAL's state after replay equals the state it would have had if every posting had gone to it directly.

23.4 Rehearsals and communication

Each of the three rehearsals runs the full timeline against a production-sized copy of CORAL's data in the pre-production environment, with product system simulators replaying a recorded Friday evening of traffic. Each rehearsal produces a timing report, and any step that overruns by more than 15 minutes is redesigned before the next rehearsal.

Customers are told two weeks ahead that card, transfer and app services may be slower between 22:00 Friday and 06:00 Saturday. The branches and the contact centre receive a script. The regulator is informed of the cut-over date as part of the programme's regular updates.

24. Resilience and Disaster Recovery

24.1 Within the region

Every component runs across three Availability Zones. The Aurora clusters fail over to a reader in another zone in about 30 seconds; the Posting Service retries its transaction once on a failover error and otherwise returns 503 so that the caller retries with the same idempotency key. MSK tolerates the loss of one zone without data loss (`min.insync.replicas 2, acks=all`).

24.2 Region loss

The Journal Store and the Balance Store are Aurora Global Databases with secondary clusters in ap-southeast-3 (Jakarta). The ledger clusters replicate to the secondary at the storage layer asynchronously, with lag typically under one second, and on region loss the runbook promotes the secondary. EKS workloads are deployed in Jakarta at zero replicas and scaled up by the runbook; MSK in Jakarta is a separate cluster, and the Account Processors there rebuild their position from the Journal Store's outbox sequence. The runbook targets service restoration within 45 minutes, inside the one-hour RTO of NFR-3.

24.3 Testing

The region failover is exercised twice a year in a full-scale test environment and once a year in production during the maintenance window, with the CFO's approval.

24.4 Backups

Aurora automated backups are kept for 35 days, and a daily snapshot is copied to a backup vault in a separate AWS account with a vault lock. A point-in-time restore of the Journal Store to a separate cluster is tested every quarter. Because the Balance Store can be rebuilt from the Journal Store by replaying legs (Section 10), the Balance Store's backups are a convenience rather than the source of recovery.

25. Observability

Metrics: posting rate and latency by source system, rejection rate by reason, outbox lag (oldest unpublished row), Account Processor lag per partition, close step durations, reconciliation status. Alerts page the on-call engineer when outbox lag exceeds 5 seconds, any processor lag exceeds 30 seconds, or a close step exceeds its expected duration by 50 percent.

Logs are structured JSON, shipped to the bank's log platform, and carry the journal ID and source reference but never customer names or national identifiers. Traces follow a posting from the API through the outbox to the Account Processor.

Finance-facing views in the Close Console show the close's progress, the suspense account balances and the list of close adjustments.

Dashboards for the dual-run show, per business day, the reconciliation status, the number of postings copied by the dual-feed adapter, and the adapter's rejection count by reason.

25.1 Runbooks

Each alert links to a runbook. The runbooks cover a stuck outbox relay, a lagging Account Processor partition, a close step that fails, a reconciliation break, an Aurora writer failover and a full region failover. Each runbook names the role that may perform each step; no runbook step requires posting a journal, and any correction needed to the books is raised as a manual journal through the Journal Console.

26. Confirmed Decisions

ID	Decision	Rationale
DEC-01	AWS ap-southeast-1 primary, ap-southeast-3 for disaster recovery	Existing landing zone; regulator notified of the cloud arrangement in 2025
DEC-02	Aurora PostgreSQL 16 for the Journal Store and the Balance Store	Team experience; deferred constraints; Global Database
DEC-03	Amazon MSK for the event bus	Managed Kafka; IAM authentication
DEC-04	Integer minor units, ISO 4217 exponents	Principle P5
DEC-05	Five-segment chart of accounts with 1,240 accounts	Financial Control decision of June 2026
DEC-06	Business day ends at 23:30 SGT	Leaves the close two and a half hours before NFR-6
DEC-07	History migration reads CORAL's VSAM history through the Pelita Extract utility, in Phase 3 (January and February 2027)	The only supported reader for CORAL's packed-decimal history files
DEC-08	Cut-over weekend 3 and 4 April 2027	After the Q1 close; before the mainframe contract's notice date
DEC-09	Customer statements produced from the ledger . journals stream	Statements Service design v1.0

27. Pending Backlog

ID	Item	Owner	Due
PB-01	Confirm the regulatory mapping changes for the new chart of accounts with the reporting tool's configuration team	Regulatory Reporting	2026-11-30
PB-02	Renew the Pelita Extract licence, which ends on 31 December 2026; renewal for 2027 requested from Rambai Software, no reply yet	Procurement	Open
PB-03	Decide whether loan arrears ageing moves from the lending system to Selasih	Lending	2027-01-31
PB-04	Agree the format of the daily trial balance file with Group Finance	Financial Reporting	2026-12-15
PB-05	Load test the Balance Store at the payday peak with production-shaped data	Core Banking Engineering	2027-01-15

28. Validation and Acceptance Criteria

28.1 Functional

Requirement	Acceptance test
FR-1	A test suite of 500 journal shapes, including unbalanced ones, is posted; every unbalanced request is rejected and every accepted journal balances per currency. A direct database insert of an unbalanced journal is aborted by the deferred trigger.
FR-2	10,000 requests are sent twice concurrently with the same keys; exactly 10,000 journals exist, and reused keys with different bodies return 409.
FR-4	Accrual for 1,000 sample accounts over one month matches an independent spreadsheet calculation to the minor unit.
FR-5	Revaluation of the month-end test positions matches Financial Control's worked example.
FR-8	A maker cannot approve their own journal; a journal above SGD 1,000,000 cannot post with one approval.
FR-9	Section 21.3 validation passes with zero differences.
FR-10	Twenty consecutive matched days (Section 22.2).
FR-11	Rollback rehearsal completed with CORAL's post-replay trial balance equal to Selasih's.
FR-12	Internal Audit's tool verifies the hash chain for the test period; an altered record is detected.

28.2 Non-functional

Requirement	Acceptance test
NFR-1	A one-hour load test at 1,000 journals per second with production-shaped journals shows a p99 of 150 ms or less.
NFR-2	Measured over the first three months of production.
NFR-3	The production failover exercise restores posting within one hour, and every journal acknowledged before the failure is present after it.
NFR-4	A sample of 20 archived records between 13 months and seven years old is retrieved within four hours, and a sample of 5 records older than seven years within 48 hours.
NFR-6	Close completes by 02:00 SGT on every dual-run day.
NFR-7	Access review of role assignments and a test that a maker cannot approve.
NFR-8	As FR-12.

29. Implementation Readiness Assessment

Area	Status	Notes
Journal model and double entry	Ready	Section 7
Posting API and idempotency	Ready	Section 8
Account processing	Ready	Prototype measured at 180 ms p99 lag
Event bus and outbox	Ready	Section 10

Area	Status	Notes
Interest accrual and revaluation	Ready	Worked examples agreed with Financial Control
End-of-day close	Ready	Close steps timed in the prototype
Regulatory reporting extracts	Needs work	Depends on PB-01
Manual journals and access	Ready	Roles agreed with Financial Control
Audit and retention	Ready	Sections 18 and 19
History migration	Ready	First extract sample loaded
Dual-run and cut-over	Ready	Rehearsal plan agreed
Disaster recovery	Ready	Runbook drafted

30. Build Phases

Phase	Dates	Content
1. Foundations	Oct to Dec 2026	Journal Store, Posting API, outbox, Account Processors, consoles
2. Finance functions	Nov 2026 to Jan 2027	Accrual, revaluation, close, reporting extracts
3. History migration	Jan to Feb 2027	Pelita Extract runs, loader, validation, rehearsals
4. Dual-run	1 Feb to 2 Apr 2027 (9 weeks)	Dual-feed adapter live, daily reconciliation, go/no-go on 1 Apr
5. Cut-over	3 to 4 Apr 2027	Section 23
6. Hypercare	Apr to Jun 2027	CORAL read-only; decommissioning plan